UNIVERSIDAD DE EL SALVADOR EN LÍNEA

EDUCACIÓN A DISTANCIA

TAREA DE INVESTIGACIÓN APLICADA

FACULTAD DE INGENIERÍA Y ARQUITECTURA

ESCUELA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROGRAMACIÓN ORIENTADA A OBJETOS

POO-115

TEMA DE INVESTIGACIÓN

Clave 6. Herencia y polimorfismo en el diseño de sistemas

PRESENTADO POR

NOMBRE	CARNET
Ángel Gabriel Hércules Hércules	*HH26008*
Tania Pamela Elías Vega	*EV26004*
Kevin Javier Vásquez Girón	*VG26013*
Roberto Valentín Fernández Leiva	*FL26003*

GRUPO DE TUTORÍA: N.° 2 | **GRUPO DE PROYECTO** N.° 6

FECHA DE ENTREGA: 11 de octubre de 2026

DOCENTE: Ing.JoséAlberto Martínez Campos

ÍNDICE.

Introducción..... 4
Objetivos..... 5
Objetivo General..... 5
Objetivos Específicos..... 5
Marco Teórico..... 6
Herencia y polimorfismo en el diseño de sistemas orientados a objetos..... 6
1. Conceptos Fundamentales..... 6
1.1. Clases y Objetos..... 6
1.2. Herencia..... 7
1.3. Polimorfismo..... 7
2. Principios Relacionados con la Herencia y el Polimorfismo..... 8
2.1. Relación "es-un"..... 8
2.2. Sustituibilidad y Subtipado..... 8
2.3. Encapsulamiento y Herencia..... 9
3. Elementos y Componentes Relevantes..... 9
4. Ventajas de la Herencia y el Polimorfismo..... 11
5. Desventajas y Riesgos..... 11
6. Aplicaciones en el Diseño de Software..... 12
7. Ejemplos para la Comprensión de los Conceptos..... 13
8. Consideraciones para un Uso Adecuado..... 14
Desarrollo del Caso de Estudio..... 15
1. Descripción y Contextualización del Problema..... 15
2. Identificación de la Necesidad..... 15
3. Descripción de la Solución Propuesta..... 16
4.Requerimientos Funcionales y No Funcionales..... 16
5.Identificación de los Actores o Usuarios del Sistema..... 18
6.Descripción de las Principales Funcionalidades..... 18
7. Explicación de Cómo los Conceptos Investigados Serán Aplicados en el Caso de Estudio..... 19
DISEÑO DE LA SOLUCIÓN MEDIANTE UML..... 21
1. Diagrama de Casos de Uso..... 21
1.1 Propósito..... 21
1.2 Elementos del Diagrama..... 21
2. Diagrama de Clases..... 22
2.1 Propósito..... 22
2.2 Elementos del Diagrama..... 23
3. Coherencia entre los Diagramas y con la Implementación..... 26
Implementación en Java..... 28
1. Descripción de la Implementación..... 28
2. Arquitectura Utilizada..... 28
3. Principales Clases, Componentes Y Relaciones..... 29
4. Aplicación de los Conceptos Investigados en el Código..... 30
Pruebas y Resultados de las Pruebas Realizadas..... 32

1.Casos de Prueba Representativos y Resultados Obtenidos..... 32
2.Evidencia Del Funcionamiento Correcto De Las Principales Funcionalidades.....34
Conclusiones.....39
Principales conocimientos adquiridos..... 39
Cumplimiento de los objetivos planteados.....40
Resultados obtenidos mediante la aplicación de los conceptos investigados..... 40
Reflexión sobre la importancia de los conceptos estudiados en el desarrollo de soluciones de software.....40
Referencias Bibliográficas..... 42
Anexos..... 44
Enlace al video de exposición y demostración del caso de estudio.....44

Introducción

En el ámbito del diseño de sistemas orientados a objetos, la herencia y el polimorfismo constituyen pilares fundamentales para la creación de software modular, extensible y mantenible. La herencia permite definir una nueva clase a partir de clases existentes, facilitando la reutilización de código y estableciendo relaciones jerárquicas entre componentes. Por su parte, el polimorfismo otorga a los objetos la capacidad de responder a un mismo mensaje de diferentes maneras según su contexto, permitiendo una mayor adaptabilidad en la ejecución del sistema.

La importancia de investigar y aplicar adecuadamente estos conceptos radica en su impacto sobre la arquitectura y calidad del software. Sin embargo, su uso requiere un diseño cuidadoso para balancear la herencia con el encapsulamiento y prevenir complejidades innecesarias en el mantenimiento y las pruebas del sistema.

En el desarrollo de software moderno, especialmente en lenguajes como Java, la herencia y el polimorfismo se aplican directamente para resolver problemas complejos del mundo real mediante abstracciones sólidas. Permiten diseñar soluciones escalables donde múltiples tipos de entidades comparten comportamientos comunes, pero responden de forma específica según sus requerimientos particulares, desacoplando la lógica de control del comportamiento individual de cada objeto.

Este documento aborda los fundamentos teóricos de la herencia y el polimorfismo, analizando sus definiciones, ventajas y principios de diseño asociados. Asimismo, se presenta el desarrollo de un caso de estudio enfocado en un sistema de administración de empleados, su modelado mediante diagramas UML, su correspondiente implementación en Java y la evaluación de resultados y conclusiones obtenidas

Objetivos

Objetivo General.

Investigar, analizar y aplicar los principios de herencia y polimorfismo en el diseño y desarrollo de un sistema de administración de empleados en Java, con el fin de estructurar un software modular, extensible y mantenible que optimice la reutilización de código y cumpla con los principios fundamentales de la programación orientada a objetos.

Objetivos Específicos.

- *Analizar los fundamentos teóricos y principios de diseño asociados a la herencia y el polimorfismo para comprender su impacto en la calidad, modularidad y mantenibilidad del software.*
- *Diseñar el modelo de la solución para el sistema de administración de empleados mediante diagramas UML (casos de uso y clases), asegurando la coherencia con los requerimientos del caso de estudio*
- *Implementar el sistema en Java aplicando jerarquías de clases y comportamiento polimórfico, evaluando su correcta ejecución a través de casos de prueba representativos.*

Marco Teórico.

Herencia y polimorfismo en el diseño de sistemas orientados a objetos

La programación orientada a objetos constituye un enfoque de desarrollo de software basado en la representación de los elementos de un sistema mediante objetos que poseen datos y comportamientos. Dentro de este paradigma, la herencia y el polimorfismo constituyen dos mecanismos fundamentales para desarrollar sistemas reutilizables, extensibles y mantenibles. La herencia permite establecer relaciones entre clases y reutilizar características previamente definidas, mientras que el polimorfismo permite que objetos de diferentes tipos respondan de manera particular ante una misma operación (Danforth & Tomlinson, 1998)(Meyer, 1996, 105-108)(Saini, 2009, 1-15)

1. Conceptos Fundamentales.

1.1. Clases y Objetos.

Una clase puede entenderse como una plantilla que define las características y comportamientos que tendrán los objetos creados a partir de ella. Los objetos, por su parte, son instancias concretas de dichas clases (Danforth & Tomlinson, 1998). En el diseño orientado a objetos, las clases pueden organizarse mediante jerarquías que permiten representar relaciones entre conceptos.

Por ejemplo, puede definirse una clase general denominada Vehículo, que contenga características comunes como ruedas y motor. A partir de ella pueden definirse clases más específicas, como Coche y Camión, que incorporen características propias sin necesidad de volver a definir las características comunes.

1.2. Herencia.

La herencia es un mecanismo mediante el cual una clase nueva, denominada subclase o clase derivada, se define a partir de una clase existente, denominada superclase o clase base. La subclase recibe las características y comportamientos de la superclase y puede incorporar nuevos elementos o modificar los heredados (Danforth & Tomlinson, 1998) (Meyer, 1996).

Una característica esencial de la herencia es la reutilización de código. En lugar de implementar nuevamente características que ya existen en una clase, estas pueden heredarse de una clase base. Además, una subclase puede sobrescribir determinados métodos para proporcionar un comportamiento específico (Saini, 2009).

Meyer identifica doce modalidades de herencia agrupadas en tres categorías principales: herencia de modelo, herencia de software y herencia de variación (Meyer, 1996). La primera representa relaciones entre conceptos del modelo, la segunda expresa relaciones relacionadas con la estructura del software y la tercera describe una clase según la forma en que se diferencia de otra.

1.3. Polimorfismo.

El polimorfismo significa "muchas formas" y representa la capacidad de una misma operación de aplicarse a diferentes tipos de objetos, permitiendo que cada uno responda de acuerdo con su comportamiento particular (Danforth & Tomlinson, 1998).

En el diseño orientado a objetos, el polimorfismo puede presentarse de diferentes maneras. Entre ellas se encuentran el polimorfismo ad hoc, relacionado con la sobrecarga; el polimorfismo paramétrico, en el cual una función puede actuar de manera uniforme independientemente del tipo de sus argumentos; y el polimorfismo de inclusión o de subtipos, relacionado con objetos pertenecientes a tipos vinculados mediante relaciones de inclusión (Danforth & Tomlinson, 1998).

Dos elementos importantes para implementar el polimorfismo mediante herencia son la sobrescritura de métodos y el enlace dinámico. La sobrescritura permite que una clase hija redefina un método heredado, mientras que el enlace dinámico determina durante la ejecución qué implementación concreta del método debe utilizarse (Saini, 2009).

2. Principios Relacionados con la Herencia y el Polimorfismo

2.1. Relación "es-un"

Uno de los principios fundamentales para utilizar correctamente la herencia consiste en que debe existir una relación conceptual de "es-un" entre la subclase y la superclase (Meyer, 1996). Por ejemplo, un coche es un vehículo, por lo que puede existir una relación de herencia entre ambas clases.

Esta relación permite distinguir la herencia de la composición. Cuando un objeto contiene o utiliza otro objeto, puede existir una relación de "tiene-un" en lugar de una relación de "es-un". Una tarta de manzana, por ejemplo, tiene una manzana como parte de sus componentes, pero no es una manzana (Meyer, 1996).

Meyer identifica tres usos incorrectos frecuentes de la herencia: confundir una relación "tiene-un" con una relación "es-un", crear jerarquías innecesariamente complejas mediante la denominada taxomanía y utilizar la herencia únicamente para reutilizar características sin que exista una relación conceptual adecuada (Meyer, 1996).

2.2. Sustituibilidad y Subtipado

La herencia y el subtipado son conceptos relacionados, pero no idénticos. La herencia permite principalmente reutilizar una implementación, mientras que el subtipado se relaciona con la posibilidad de utilizar objetos de un tipo determinado en contextos donde se espera un tipo más general (Danforth & Tomlinson, 1998).

El principio de sustituibilidad establece que, cuando un tipo es subtipo de otro, los objetos del subtipo deben poder utilizarse en los contextos donde se espera el tipo superior sin alterar el comportamiento esperado (Danforth & Tomlinson, 1998). Esto permite que una función diseñada para trabajar con objetos de tipo Vehículo pueda recibir objetos de tipo Coche o Camión, siempre que estos cumplan con el comportamiento esperado.

2.3. Encapsulamiento y Herencia

El encapsulamiento busca ocultar los detalles internos de una clase y controlar la forma en que sus datos y comportamientos pueden ser utilizados. Sin embargo, la herencia puede generar una tensión con este principio debido a que una subclase puede necesitar acceder a elementos definidos en su superclase (Danforth & Tomlinson, 1998), (Kilian, 1991).

Se señala que la introducción de la herencia puede comprometer los beneficios del encapsulamiento cuando las subclases dependen directamente de la representación interna de las superclases (Snyder, 1986). Para disminuir este problema se plantea utilizar operaciones para acceder a los datos, controlar la visibilidad de los elementos y mantener una separación entre el subtipado y la herencia (Snyder, 1986).

También se destaca la importancia de mantener las variables de instancia privadas y limitar la cantidad de decisiones de implementación que son expuestas mediante los métodos públicos (Robillard, 2022).

3. Elementos y Componentes Relevantes

En el diseño de sistemas orientados a objetos relacionados con la herencia y el polimorfismo intervienen diferentes elementos:

- Clases: representan plantillas que definen datos y comportamientos.

- Objetos: constituyen instancias concretas de las clases (Danforth & Tomlinson, 1998).
- Atributos: representan los datos o características asociados con los objetos (Danforth & Tomlinson, 1998).
- Métodos: representan los comportamientos u operaciones que pueden realizar los objetos (Danforth & Tomlinson, 1998).
- Superclases: clases generales de las cuales otras clases pueden heredar.
- Subclases: clases derivadas que reciben características de una superclase y pueden agregar o modificar comportamientos.
- Clases abstractas: clases incompletas diseñadas para ser heredadas y que no están destinadas a ser instanciadas directamente (Meyer, 1996).
- Sobrescritura: mecanismo mediante el cual una subclase redefine un método heredado (Saini, 2009).
- Enlace dinámico: mecanismo mediante el cual se determina durante la ejecución qué implementación concreta de un método será ejecutada (Saini, 2009).
- Jerarquías de clases: estructuras que organizan las relaciones de generalización y especialización entre clases.
- Encapsulamiento: mecanismo relacionado con el control del acceso a los datos y detalles internos de los objetos (Danforth & Tomlinson, 1998)(Kilian, 1991).
- Acoplamiento y cohesión: conceptos relacionados con la dependencia entre clases y con el grado en que las responsabilidades de una clase están relacionadas (Henry & Gotterbarn, 1996)(Kilian, 1991).

La adecuada organización de estos elementos permite desarrollar estructuras de software en las que las clases puedan reutilizar características y especializar comportamientos sin generar dependencias innecesarias.

4. Ventajas de la Herencia y el Polimorfismo

La utilización adecuada de la herencia y el polimorfismo puede proporcionar diversas ventajas en el desarrollo de sistemas orientados a objetos.

Una de las principales es la reutilización de código, debido a que las características de una clase base pueden ser aprovechadas por diferentes clases derivadas. Esto evita repetir implementaciones y puede reducir el tamaño del código (Riehle, 1997).

Otra ventaja es la extensibilidad. La organización mediante generalización y especialización facilita la incorporación de nuevas clases y comportamientos sin tener que reconstruir completamente las estructuras existentes (Danforth & Tomlinson, 1998).

El polimorfismo también proporciona flexibilidad durante la ejecución, ya que el enlace dinámico permite que una misma operación produzca comportamientos diferentes dependiendo del tipo real del objeto (Saini, 2009).

Además, determinados patrones de diseño utilizan estos mecanismos para separar responsabilidades y facilitar la modificación de partes de un sistema. Entre ellos se encuentran Iterator, Factory Method, Composite, Bridge, Observer, Mediator y Chain of Responsibility (Riehle, 1997)(Kobylinski et al., 1997).

Los patrones de diseño también pueden combinarse para formar estructuras más complejas. El patrón MVC, por ejemplo, permite separar diferentes responsabilidades relacionadas con el modelo, la vista y el controlador (Riehle, 1997)(Kobylinski et al., 1997).

5. Desventajas y Riesgos

A pesar de sus ventajas, la herencia y el polimorfismo también presentan riesgos cuando se utilizan de manera incorrecta.

Uno de ellos es el acoplamiento excesivo. Cuando diferentes clases dependen fuertemente unas de otras, disminuye su posibilidad de reutilización independiente. Una dependencia excesiva puede provocar que modificar o reutilizar una clase implique trasladar también otras partes del sistema (Kilian, 1991).

Otro riesgo corresponde a las jerarquías de herencia mal diseñadas. La taxomanía, la herencia por conveniencia y la confusión entre las relaciones "es-un" y "tiene-un" pueden producir estructuras complejas y difíciles de mantener (Meyer, 1996).

También existe una fragilidad ante los cambios, debido a que una modificación realizada en una clase base puede afectar a las clases derivadas que dependen de ella.

La herencia puede además generar un conflicto con el encapsulamiento, especialmente cuando las clases derivadas dependen directamente de detalles internos de las clases base (Danforth & Tomlinson, 1998)(Kilian, 1991)(Snyder, 1986).

El polimorfismo también puede dificultar las pruebas de software. Debido al enlace dinámico, el tipo exacto del objeto y la implementación que será ejecutada pueden no conocerse hasta el momento de ejecución. Esto puede hacer menos transparente el flujo de control y dificultar la identificación de determinadas anomalías (Saini, 2009).

Para enfrentar estos problemas, Saini plantea utilizar técnicas de prueba a nivel de diseño, incluyendo diagramas de secuencia UML, con el objetivo de identificar posibles anomalías relacionadas con el polimorfismo y la sobrescritura (Saini, 2009).

6. Aplicaciones en el Diseño de Software

La herencia y el polimorfismo tienen aplicación en diferentes aspectos del diseño orientado a objetos. Uno de ellos es la construcción de jerarquías de clases, donde una clase general puede proporcionar características comunes y las clases especializadas pueden añadir o modificar comportamientos.

También son utilizados en diferentes patrones de diseño. El patrón Iterator separa las operaciones relacionadas con el acceso a los elementos de una colección del objeto que almacena dichos elementos. Factory Method proporciona mecanismos para la creación de objetos y puede generar jerarquías relacionadas. Composite permite representar estructuras jerárquicas en forma de árbol, mientras que Bridge busca desacoplar una abstracción de sus implementaciones (Riehle, 1997)(Kobylinski et al., 1997).

Otros patrones como Observer, Mediator y Chain of Responsibility permiten coordinar objetos mediante diferentes mecanismos relacionados con eventos y comunicación entre componentes (Riehle, 1997).

El uso de patrones durante las etapas de análisis y diseño también puede contribuir a reducir el tiempo de diseño y el riesgo de errores.

7. Ejemplos para la Comprensión de los Conceptos

Un ejemplo básico de herencia puede representarse mediante las clases Vehículo, Coche y Camión. La clase Vehículo puede contener características comunes, mientras que Coche y Camión pueden especializar esas características incorporando elementos propios.

En este mismo ejemplo puede observarse el polimorfismo. Si Vehículo define una operación como avanza, tanto Coche como Camión pueden proporcionar una implementación específica. Cuando se solicita la operación sobre un objeto concreto, el enlace dinámico permite ejecutar la implementación correspondiente al tipo real del objeto (Saini, 2009).

Otro ejemplo es el de Punto y PuntoColoreado. La clase Punto puede contener coordenadas, mientras que PuntoColoreado puede heredar esas coordenadas y agregar un color. En este caso se representa una relación de especialización donde el segundo concepto incorpora las características del primero y agrega una característica adicional.

La relación entre Elipse y Círculo permite observar otro caso de especialización. Un círculo puede considerarse una elipse con una condición particular relacionada con sus focos (Meyer, 1996).

Por otra parte, el ejemplo de Profesor y Curso permite comprender el problema del acoplamiento. Si ambas clases mantienen referencias mutuas, cada una puede depender de la otra, disminuyendo su independencia y reutilización (Kilian, 1991).

Finalmente, el ejemplo de un Diccionario que hereda de Conjunto en Smalltalk permite observar una situación en la que la herencia puede utilizarse por conveniencia de implementación, aunque no exista una relación conceptual adecuada entre ambos conceptos (Danforth & Tomlinson, 1998).

8. Consideraciones para un Uso Adecuado

La herencia debe utilizarse cuando exista una relación conceptual clara de "es-un" entre la clase derivada y la clase base. Cuando el objetivo principal sea únicamente reutilizar código y no exista dicha relación, la composición puede constituir una alternativa que mantenga las piezas del sistema más independientes y reutilizables (Kilian, 1991).

Asimismo, el uso del polimorfismo debe considerar el comportamiento esperado de los objetos. Si una clase derivada sustituye a una clase base, debe mantener el comportamiento que se espera del tipo general. De esta manera, la combinación entre herencia, subtipado, sobrescritura y enlace dinámico puede utilizarse para construir sistemas flexibles sin perder la coherencia del diseño (Danforth & Tomlinson, 1998)(Saini, 2009).

En consecuencia, la herencia y el polimorfismo no deben considerarse únicamente como mecanismos para reducir código repetido. Su utilización requiere analizar las relaciones entre las clases, el nivel de acoplamiento, el encapsulamiento y el comportamiento esperado de los objetos. Un diseño adecuado permite aprovechar la

reutilización y extensibilidad, mientras que un diseño inadecuado puede producir jerarquías rígidas, dependencias excesivas y dificultades de mantenimiento (Danforth & Tomlinson, 1998)(Meyer, 1996)(Kilian, 1991).

Desarrollo del Caso de Estudio.

1. Descripción y Contextualización del Problema

Una organización con empleados que cobran de formas distintas necesita calcular y controlar su nómina. En el caso estudiado conviven tres modalidades de pago: empleados administrativos con salario mensual fijo, empleados por hora que cobran según las horas trabajadas y empleados por comisión que reciben un salario base más un porcentaje de sus ventas. Cuando la nómina se lleva de forma manual o con hojas de cálculo sueltas, cada modalidad exige una fórmula distinta, y es fácil cometer errores de cálculo, duplicar registros o perder el control del total que la empresa debe pagar.

Desde el punto de vista del desarrollo de software, el problema es también un ejemplo clásico de modelado orientado a objetos: varias entidades comparten datos y comportamientos comunes (identificador, nombre, cálculo de salario, visualización), pero difieren en la regla concreta con que se calcula el pago. Resolverlo bien implica elegir una estructura que evite repetir código y que permita agregar nuevos tipos de empleado sin reescribir el sistema.

2. Identificación de la Necesidad

- *Se necesita una herramienta que:*
- *Centralice el registro de todos los empleados en un solo lugar.*
- *Calcule automáticamente el salario de cada uno según su tipo de contrato.*
- *Permita corregir datos (nombre, horas, ventas, salario) sin volver a registrar al empleado.*

- *Permita consultar a los empleados por categoría y obtener un reporte consolidado de la nómina con el total a pagar.*
- *Sea sencilla de usar por una persona no técnica, mediante un menú interactivo.*

3. Descripción de la Solución Propuesta

Se propone el Sistema de Nómina, una aplicación de consola en Java con un menú interactivo. Su núcleo es una jerarquía de clases: la clase abstracta ***Empleado*** *define los datos comunes y el contrato que toda subclase debe cumplir* ***(calcularSalario()*** *y* ***actualizarDatos()****), y tres subclases concretas* ***(EmpleadoAdministrativo, EmpleadoPorHora, EmpleadoPorComision)*** *implementan cada regla de pago. La clase* ***AdminEmpleados*** *actúa como gestor: guarda todos los empleados en una misma lista de tipo* ***ArrayList<Empleado>*** *y ofrece las operaciones de registro, modificación, consulta por tipo y reporte. La clase* ***Main*** *presenta el menú y delega cada opción en* ***AdminEmpleados****.*

La decisión de diseño más importante es tratar a todos los empleados como el tipo base ***Empleado****. Así, el gestor calcula salarios y muestra datos sin preguntar de qué tipo es cada uno: el polimorfismo elige la implementación correcta en tiempo de ejecución.*

4.Requerimientos Funcionales y No Funcionales.

Tabla 1.

Descripción de requerimientos funcionales/no funcionales.

Funcional/ No funcional	Requerimiento	Descripción

FUNCIONAL	Registrar empleado	Solicitar ID, nombre y tipo (administrativo, por hora o por comisión) y capturar los datos propios del tipo elegido.
FUNCIONAL	Calcular salario	Obtener el salario según el tipo: mensual fijo; horas × valor por hora; salario base + (ventas × porcentaje de comisión).
FUNCIONAL	Modificar información	Buscar un empleado por ID y actualizar su nombre y sus datos específicos; presionar ENTER conserva el valor actual.
FUNCIONAL	Consultar por tipo	Mostrar solo los empleados de la categoría elegida y el total de registros encontrados
FUNCIONAL	Generar reporte de nómina	Listar cada empleado con ID y salario, contar los empleados por tipo y mostrar el pago total.
FUNCIONAL	Mostrar datos	Presentar los datos de cada empleado, incluidos los específicos de su tipo y su salario calculado.
FUNCIONAL	Evitar duplicados	Rechazar un empleado cuyo ID ya exista.
FUNCIONAL	Validar entradas del menú	Rechazar opciones no numéricas o fuera del rango 1–5 y avisar cuando no hay empleados registrados.
NO FUNCIONAL	Usabilidad	interfaz de consola en español, con menú numerado, mensajes claros y opción de conservar valores con ENTER

NO FUNCIONAL	Mantenibilidad	Código dividido en clases con una sola responsabilidad y documentado con Javadoc.
NO FUNCIONAL	Extensibilidad	Un nuevo tipo de empleado se agrega creando una subclase, sin modificar la lógica de cálculo existente.
NO FUNCIONAL	*Portabilidad*	*Funciona en cualquier sistema con una JVM; no depende de librerías externas.*
NO FUNCIONAL	*Rendimiento*	*Datos en memoria con búsqueda lineal por ID; adecuado para cientos de empleados.*
NO FUNCIONAL	*Confiabilidad*	*Validación básica de las opciones del menú; los datos no se conservan al cerrar el programa.*

5.Identificación de los Actores o Usuarios del Sistema.

- ***Administrador de nómina (Actor principal y usuario principal):*** *Es la persona encargada de operar el menú. Registra empleados, corrige sus datos, consulta por tipo y genera el reporte.*
- ***Empleado (entidad, no actor)****: sus datos son los registrados y consultados, pero no interactúa con el sistema.*

6.Descripción de las Principales Funcionalidades.

- **Registrar nuevo empleado:** pide ID y nombre, pregunta el tipo, crea el objeto con valores en cero y llama a ***actualizarDatos()*** para capturar los datos propios del tipo.

- **Modificar información:**recibe un ID, localiza al empleado, permite cambiar el nombre y luego delega la actualización de sus datos específicos en el propio objeto.
- **Consultar por tipo:** muestra un submenú con las tres categorías y filtra la lista comparando el nombre de la clase de cada objeto.
- **Generar reporte general:** recorre la lista, imprime nombre, ID y salario de cada empleado, acumula el total y cuenta cuántos hay de cada tipo.
- **Salir del sistema:** termina el ciclo del menú.

7. Explicación de Cómo los Conceptos Investigados Serán Aplicados en el Caso de Estudio.

Tabla 2.

Relación entre conceptos y planificación del caso de estudio.

Concepto	***Planificación de aplicación en el caso de estudio***
Herencia	Se planea definir una clase base común para los empleados y hacer que las distintas modalidades de contratación (administrativo, por hora y por comisión) extiendan de ella para reutilizar atributos básicos y métodos compartidos.

Polimorfismo	Se prevé que el gestor administre todos los registros desde una colección uniforme referenciada por el tipo base, de modo que la invocación del cálculo de salario determine de forma dinámica la regla correspondiente según cada tipo.
Abstracción	Se planifica modelar la clase base como una abstracción que defina la estructura general del empleado y los métodos obligatorios (como el cálculo de salario), evitando que pueda ser instanciada por sí sola.
Sobrescritura	Se planea que cada tipo de empleado específico redefina las operaciones clave del cálculo salarial y despliegue de información para adaptar las operaciones abstractas a la lógica propia de cada contrato.
Encapsulamiento	Se contempla proteger el acceso directo a los campos de datos asignándoles visibilidad privada, permitiendo su lectura y modificación únicamente a través de métodos de acceso (getters y setters) y operaciones controladas.
Superclase	Se proyecta que el constructor de la superclase reciba y centralice la inicialización de las propiedades esenciales durante el proceso de instanciación de las subclases.

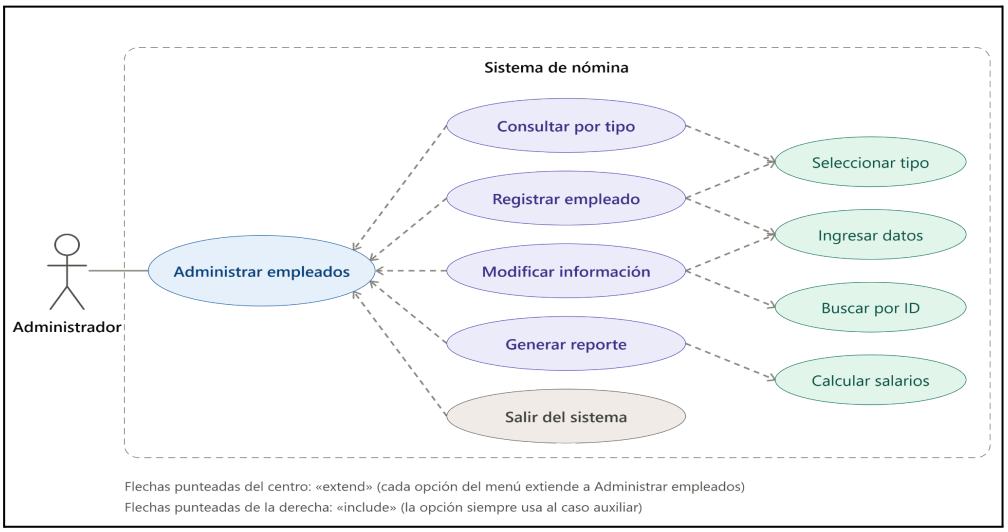
DISEÑO DE LA SOLUCIÓN MEDIANTE UML

1. Diagrama de Casos de Uso

1.1 Propósito

El diagrama de uso delimita el alcance del sistema, identifica a quien lo utiliza y enumera las funcionalidades que ofrece. Sirve de puente entre los requerimientos funcionales de la sección anterior y el diseño de clases.

Figura 1. Diagrama de sistema de nómina.



Nota.Generado con Google Gemini, 9 de octubre de 2026, a partir de la solicitud de crear imagen de diagrama según el Script PlantUML y colocarle colores.

1.2 Elementos del Diagrama

- **Actor - Administrador:** única persona que interactúa con el sistema; es quien registra, modifica, consulta y reporta la información de los empleados desde el menú de consola.

- **Límite del sistema - «Sistema de nómina»:** el rectángulo punteado encierra las funcionalidades que forman parte de la aplicación.
- **Casos de uso Principal - Administrar empleados:** representa el menú interactivo, punto de entrada de todas las aplicaciones. Es el único asociado directamente al actor.
- **Casos de uso de las opciones del menú (color morado):** registrar empleado, modificar información, consultar por tipo, generar reporte y salir del sistema. Correspondiente a las cinco opciones numeradas del menú (1 a 5).
- **Casos de uso auxiliares (color verde):** Seleccionar tipo, ingresar datos, buscar por ID y calcular salarios. Son pasos reutilizables que las opciones del menú siempre ejecutan; por ejemplo, tanto registrar empleado como consultar por tipo necesitan seleccionar tipo.
- **Relaciones:** las flechas punteadas del centro (opción → administrar empleados) se modelaron como «extend», y las de la derecha (opción → casó auxiliar) como «include».

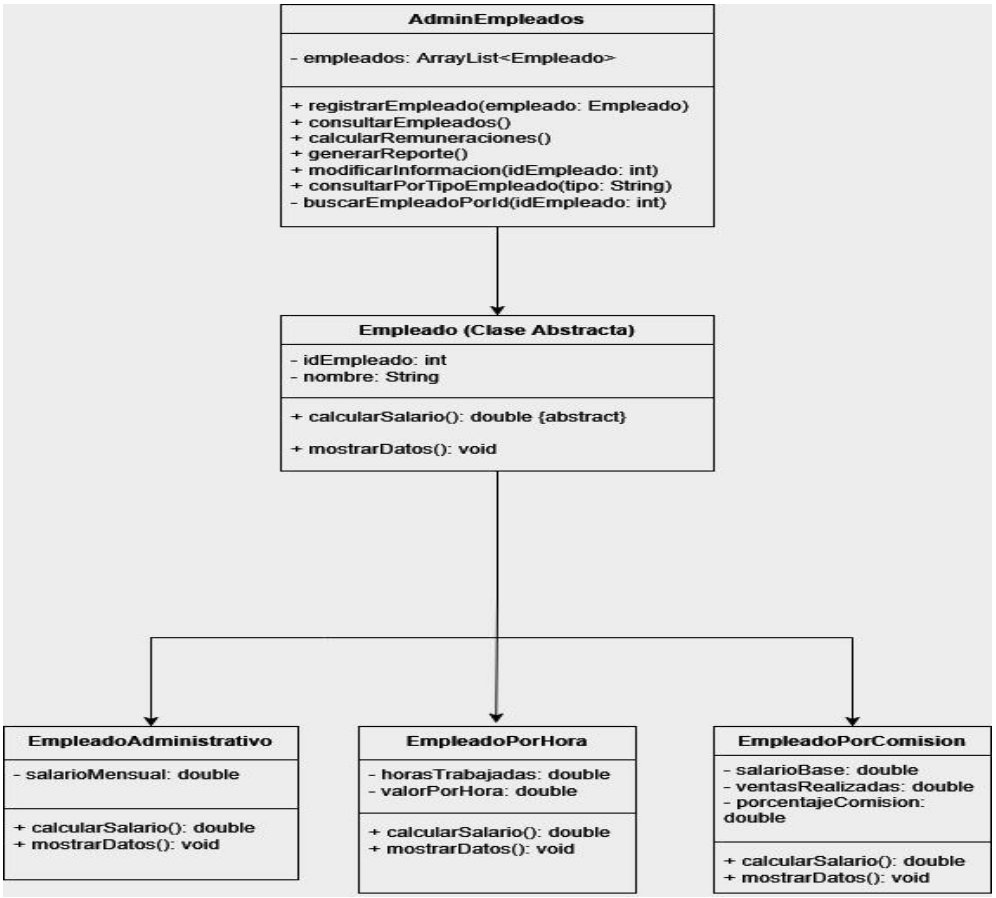
2. Diagrama de Clases

2.1 Propósito

El diagrama de clases muestra la estructura estática de la solución: las clases, sus atributos y operaciones, y las relaciones entre ellas. Es el diagrama donde se hace visible la aplicación de los conceptos investigados: una **Clase abstracta** (empleado) que define el contrato común, tres **Subclases** que lo especializan mediante **herencia** y **sobreescritura,** y un gestor (AdminEmpleados) que opera sobre la jerarquía de forma **polimórfica.**

Figura 2.

Diagrama de clases para el módulo de administración de empleados.



Nota. Generado con Claude, 1 de octubre de 2026, a partir de la solicitud de diseño del sistema proporcionado.

2.2 Elementos del Diagrama

Tabla 3.

Clases del módulo de administración de empleados.

Clase	Atributo	Operaciones	Responsabilidad
AdminEmpleados	-empleados: ArrayList<empleado> - Aviso : string	+registrarEmpleado(empleado) +registrarNuevoEmpleado() +consultarEmpleados() +calcularRemuneraciones()	Gestor: almacena todos los empleados en una sola colección y

		+generarReporte() +modificarInformacion(idEmpleado) +consultarPorTipoEmpleado(tipo) -buscarEmpleadoPorId(idEmpleado)	ofrece las operaciones de negocio y solicita por consola los datos del registro y el tipo de empleado.
Empleado (abstracta)	-idEmpleado: int -nombre: string	+calcularSalario(): double {abstract} +actualizarDatos () : void {abstract} +mostrarDatos(): void getIdEmpleado() : int getNombre() : string setNombre(nombre): void	Superclase: define los datos comunes y el contrato que toda subclase debe cumplir (calcular el salario y actualizar los datos).
EmpleadoAdministrativo	-salarioMensual: double	+calcularSalario(): double +mostrarDatos() : void +actualizarDatos() : void	Salario fijo Mensual
EmpleadoPorHora	horasTrabajadas: double -valorPorHora: double	+calcularSalario(): double +mostrarDatos(): void +actualizarDatos() : void	Salario = horas trabajadas x valor por hora
EmpleadoPorComision	-salarioBase: double -ventasRealizadas: double porcentajeComision:	+calcularSalario(): double +mostrarDatos(): void +actualizarDatos() : void	Salario = base + (ventas x porcentaje de comision)

	double		

La notación de visibilidad es la estándar de UML: - indica un miembro privado y + uno público. Todos los atributos son privados, lo que respeta el encapsulamiento (Robillard, 2022) (Snyder, 1986)*; el acceso a los datos se realiza únicamente mediante las operaciones públicas.*

El método buscarEmpleadoXId() es privado porque es un detalle interno del gestor y no forma parte de su interfaz pública. La anotación {abstract} calcularSalario() y actualizarDatos() indica que la operación se declara en Empleado pero no tiene implementación allí, por lo que Empleado no puede instanciarse directamente (Meyer, 1996)

Tabla 4.

Matriz de relaciones, multiplicidad y comportamiento entre las clases del módulo de nómina.

Relación	Clases	Multiplicidad	Justificación
Generalización(herencia)	EmpleadoAdministrativo, EmpleadoPorHora y EmpleadoPorComision → Empleado	—	cumple la relación «es-un»: un empleado por hora es un empleado (Meyer, 1996).
Asociación	AdminEmpleados → Empleado	1 a 0..*	El gestor mantiene una colección ArrayList<Empleado>, para calcular salarios, listar y modificar depende del tipo base Empleado
Dependencia(crea	AdminEmpleados	—-	registrarNuevoEm

ción)	→ EmpleadoAdministrativo, EmpleadoPorHora y EmpleadoPorComision		pleado() crea con new el objeto del tipo elegido.
sobreescritura	cada subclase respecto de Empleado	—-	calcularSalario(), mostrarDatos() y actualizarDatos() se redefinen en cada subclase con su regla particular (Saini, 2009).

3. Coherencia entre los Diagramas y con la Implementación

Cada caso de uso del primer diagrama se corresponde con una operación pública de AdminEmpleados en el segundo (salvo salir del sistema, que se resuelve en el ciclo del menú de main), y cada requerimiento funcional se cubre con al menos una de ellas. Las operaciones consultarEmpleados() y calcularRemuneraciones() también están implementadas en el gestor, pero en el menú no las invoca. En la siguiente tabla se muestra su trazabilidad.

Tabla 5.

Matriz de trazabilidad entre requerimientos funcionales, casos de uso y componentes del diagrama de clases.

Requerimiento funcional	***Caso de uso***	***Elemento del diagrama de clases***
Registrar empleado/	Registrar empleado	*AdminEmpleados.registra*

evitar duplicados		*rNuevoEmpleado (), buscarEmpleadoXId () y actualizarDatos () de las tres subclases de empleado*
Modificar información	Modificar información	*AdminEmpleados.modificarInformacion (), buscarEmpleadoXId () y actualizarDatos() de las subclase*
Consultar por tipos	Consultar por tipo	*AdminEmpleados.consultarPorTipoEmpleado ()*
Calcular salario	Calcular salarios (auxiliar)	*Empleado.calcularSalario () sobreescrito en cada subclase*
Generar reporte de nóminas	Generar reporte	*AdminEmpleados.generarReporte ()*
Mostrar datos	*Generar reporte / Consultar por tipo*	*Empleado.mostrarDatos () sobreescrito en cada subclase*
Validar entradas del menu	*Administrar empleado*	*control del menu en main (no modelado como clase en el diagrama)*

Implementación en Java

1. Descripción de la Implementación

El caso de estudio se implementó como una aplicación en java (JDK 21), desarrollada con NetBeans 22, sin dependencias externas. La aplicación presenta un menú numerado en español con cinco opciones (registrar empleado, modificar información, consultar por tipo, generar reporte y salir) y almacena los datos en memoria durante la ejecución.

El proyecto, organizado en el paquete empleados, contiene las clases main, AdminEmpleados, Empleado, EmpleadoAdministrativo, EmpleadoPorHora y EmpleadoPorComision.

2. Arquitectura Utilizada

La solución se organiza en tres capas lógicas, lo que favorece la mantenibilidad y reduce el acoplamiento (Henry & Gotterbarn, 1996)[4]. La separación no es estricta, porque parte de la entrada y la salida por consola está repartida entre varias clases.

- **Presentación (main):** muestra el menú, lee la opción del teclado (y el ID en la opción 2), valida que la opción sea numérica y este entre 1 y 5, y delegada cada opción en el gestor. no contiene reglas de negocio.
- **Lógica de gestión (AdminEmpleados):** mantiene la colección de empleados y ejecuta las operaciones de registro, modificación, consulta y reporte. Además de recorrer la lista con el tipo base Empleado, solicita por consola los datos del registro y el tipo de empleado

- **Modelo de dominio (Jerarquía Empleado):** encapsula los datos de cada tipo empleado y regla con que se calcula su salario. Cada subclase también implementa mostrarDatos(), que imprime sus datos, y actualizarDatos(), que los captura por consola.

La decisión arquitectónica central es que el cálculo de salario, el listado y la modificación se apoyan del polimorfismo: AdminEmpleados recorre una lista de tipo Empleado e invoca calcularSalario(), mostrarDatos() y actualizarDatos(), y el enlace dinámico ejecuta la versión correcta (Saini, 2009)[3]. solo en tres puntos el gestor distingue el tipo concreto: el registro (registrarNuevoEmpleado() crea el objeto con new segun el tipo elegido), el filtro de consultarPorTipoEmpleado () y el conteo por tipo de generarReporte(), que comparan el nombre de la clase.

3. Principales Clases, Componentes Y Relaciones

Empleado es la superclase abstracta: declara IdEmpleado y Nombre como atributos privados, con acceso mediante métodos públicos, y declara como abstractos calcularSalario() y actualizarDatos() ; mostrarDatos ()es concreto y cada subclase lo amplía. EmpleadoAdministrativo, EmpleadoPorHora y EmpleadoPorComision () son subclases concretas que agregan los atributos propios de su tipo y sobrescriben el cálculo del salario, la presentación y la actualización de sus datos.

AdminEmpleados es el gestor: contiene el ArrayList <Empleado>, rechaza IDs duplicados al registrar y localiza empleados por ID con búsqueda lineal (buscarEmpleadoXId). Main es el punto de entrada y controla el ciclo del menú. Las relaciones entre ellas son las del diagrama de clases: generalización entre Empleado y sus

subclases, y asociación de AdminEmpleados con Empleados; además, AdminEmpleados depende de las subclases porque las crea al registrar.

4. Aplicación de los Conceptos Investigados en el Código

Tabla 6.

Principios de programación orientada a objetos aplicados en el módulo de nómina.

Concepto	***Aplicación en el caso de estudio***	***Referencia***
Herencia y relación «es-un»	*Las tres subclases extienden a Empleado (extends) y reutilizan idEmpleado y nombre sin volver a declararlos*	*Danforth & Tomlinson[1]; Meyer(1996)[2]*
Clase abstracta	*Empleado no puede instanciarse; obliga a cada subclase a implementar calcularSalario() y actualizarSalario()*	*Meyer(1996)[2]*
Sobreescritura	*Cada subclase redefine calcularSalario(), mostrarDatos() y actualizarDatos() con @Override*	*Saini (2009)[3]*
Polimorfismo de inclusión y enlace dinámico	*Una sola lista ArrayList<Empleado> contiene objetos de los tres tipos; al invocar calcularSalario () (por ejemplo, en*	*Danforth & Tomlinson[1]; Saini (2009)[3]*

	generarReporte ()) se ejecuta la versión del tipo real del objeto	
Sustituibilidad	*Cualquier subclase puede usarse donde se espera un Empleado sin alterar el cálculo, el listado ni la modificación en el gestor*	*Danforth & Tomlinson[1]*
Encapsulamiento	*Atributos privados con acceso mediante métodos públicos; las subclases no dependen de la representación interna de la superclase*	*Snyder (1986)[9]; Robillard(2022)[8]*
Extensibilidad (abierto/cerrado)	*Agregar un nuevo tipo de empleado requiere crear otras subclases; el cálculo de salario, el listado y el pago total no se modifican, Si deben actualizarse el registro, el filtro por tipo y por conteo por tipo de AdminEmpleados*	*Danforth & Tomlinson[1]*
Bajo acoplamiento	*AdminEmpleados depende de la abstracción Empleado para calcular listar y modificar; solo el registro, el filtro por tipo y por conteo por tipo depende de las subclases concretas.*	*Henry & Gotterbarn(1996)[4]; Kilian (1991)[5]*

5. **Principales Funcionalidades Implementadas**

- **Registrar empleado (opción 1):** solicita ID, nombre y tipo (administrativo, por hora o por comisión); si el ID ya existe, cancela el registro; si no, captura los datos propios del tipo elegido mediante actualizarDatos().
- **Modificar Información (opción 2):** main pide el ID y el gestor lo busca; permite cambiar el nombre y los datos específicos de cada tipo, y ENTER conserva el valor actual. Si el ID no existe lo informa
- **Consultar por tipo (opción 3):** muestra sólo empleados de la categoría elegida y el total encontrado; avisa si no hay empleados registrados o si ninguno es de ese tipo.
- **Generar reporte (opción 4):** lista a cada empleado con su ID y salario, cuenta los empleados por tipo y muestra el pago total.
- **Salir (opción 5):** muestra un mensaje de despedida, espera que se presione ENTER y termina la aplicación.
- **Validación de menú:** rechaza opciones no numéricas o fuera del rango 1-5 en el menú principal y avisa cuando no hay empleados registrados y consultar por tipo o generar reporte

Pruebas y Resultados de las Pruebas Realizadas.

1.Casos de Prueba Representativos y Resultados Obtenidos.

tabla 7. Descripción de las pruebas y resultados realizados.

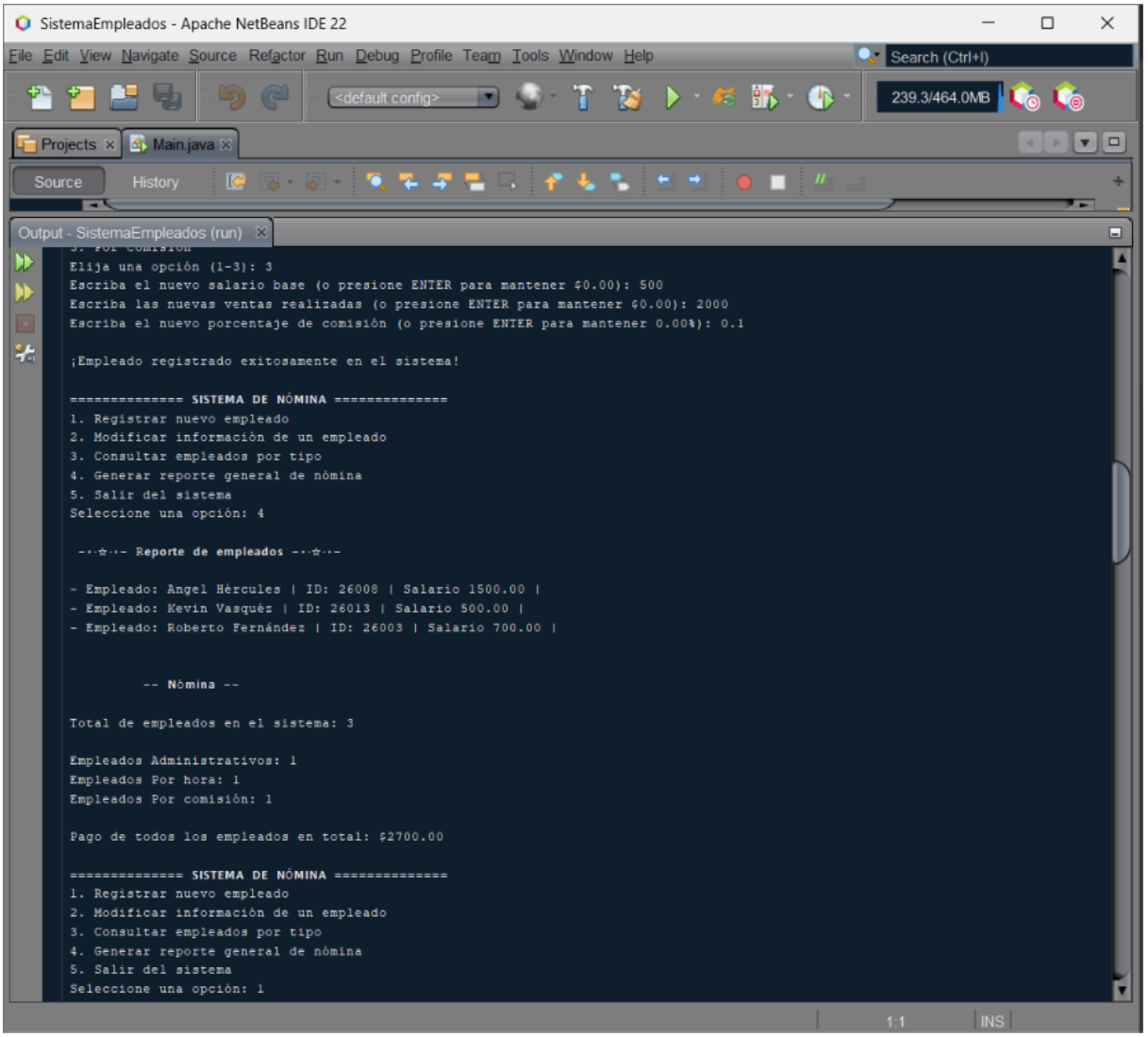
#	*Caso de Prueba*	*Resultado Esperado*	*Resultado Obtenido*	*Estado*
1	*Registrar empleado administrativo ($1500)*	*Salario: $1500.00*	*Salario 1500.00*	*Aprobado*
2	*Registrar empleado por hora (40 h × $12.5)*	*Salario: $500.00*	*Salario 500.00*	*Aprobado*

3	*Registrar empleado por comisión (base $500, ventas $2000, 0.10%)*	*Salario: $700.00*	*Salario 700.00*	*Aprobado*
4	*Reporte general con empleados registrados*	*Lista, conteo por tipo y total coherente.*	*Lista correcta con nombre, ID y salario, conteo por tipo correcto y total correcto. .*	*Aprobado*
5	*Consultar por tipo «Por Hora» (ingresaré 2 del tipo hora)*	*2 con sus respectivos datos*	*los datos de ambos empleados consultados son correctos.*	*Aprobado*
6	*Modificar empleado 2: nombre, horas = 45, ENTER en valor por hora*	*Nombre nuevo; 45 × 12.5 = 562.50; valor por hora sin cambio*	*Los datos fueron correctamente actualizados.*	*Aprobado*
7	*Modificar un ID inexistente*	*Mensaje de que no se encontró*	*- [***No se encontro un empleado con el ID***: 26000]*	*Aprobado*
8	*Opción de menú fuera de rango (9)*	*Mensaje de opción no válida*	*Opción no válida. Intente de nuevo del 1 al 5.*	*Aprobado*
9	*Opción de menú no numérica (abc)*	*Mensaje y el menú se repite*	*Escriba un número válido.*	*Aprobado*
10	*Reporte y consulta con el sistema vacío*	*Avisos de que no hay empleados*	*- [***Sistema Vacío***] Registra al menos un empleado para habilitar la f@unción de:* **generar reporte**	*Aprobado*
11	*Registrar un ID que ya existe (1)*	*Rechazar el registro duplicado*	*- [Ya existe un empleado con el ID: 26008] Registro cancelado.*	*Aprobado*
12	*Opción 5, salir*	*El programa termina*	*Saliendo del sistema - presione cualquier tecla*	*Aprobado*

2.Evidencia Del Funcionamiento Correcto De Las Principales Funcionalidades.

Figura 3.

Muestra de consola



Nota. Captura de pantalla obtenida de nuestro código

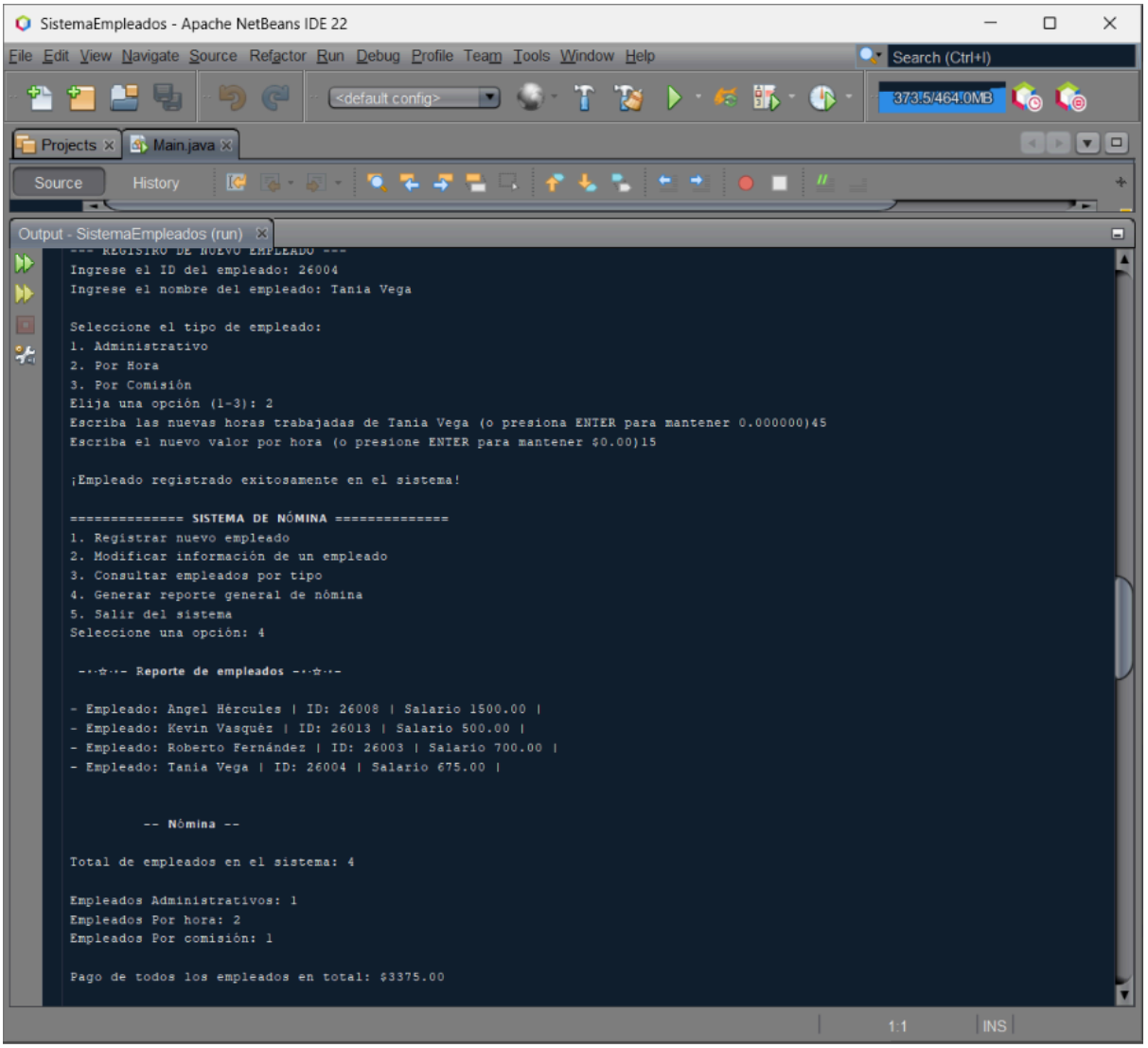
Registro de tres empleados administrativos, por hora y por comisión, y reporte general de nómina.

El reporte lista tres empleados: Angel Hércules (ID 26008) con salario 1500.00 (prueba #1), Kevin Vasquèz (ID 26013) con 500.00 (prueba #2) y Roberto Fernández (ID

26003) con 700.00 (prueba #3). El resumen indica 3 empleados en total, uno por cada tipo (administrativo, por hora y por comisión), y un pago total de $2700.00 (prueba #4).

Figura 4.

Registros de empleados como prueba.



Nota. captura de pantalla obtenida de nuestro código.

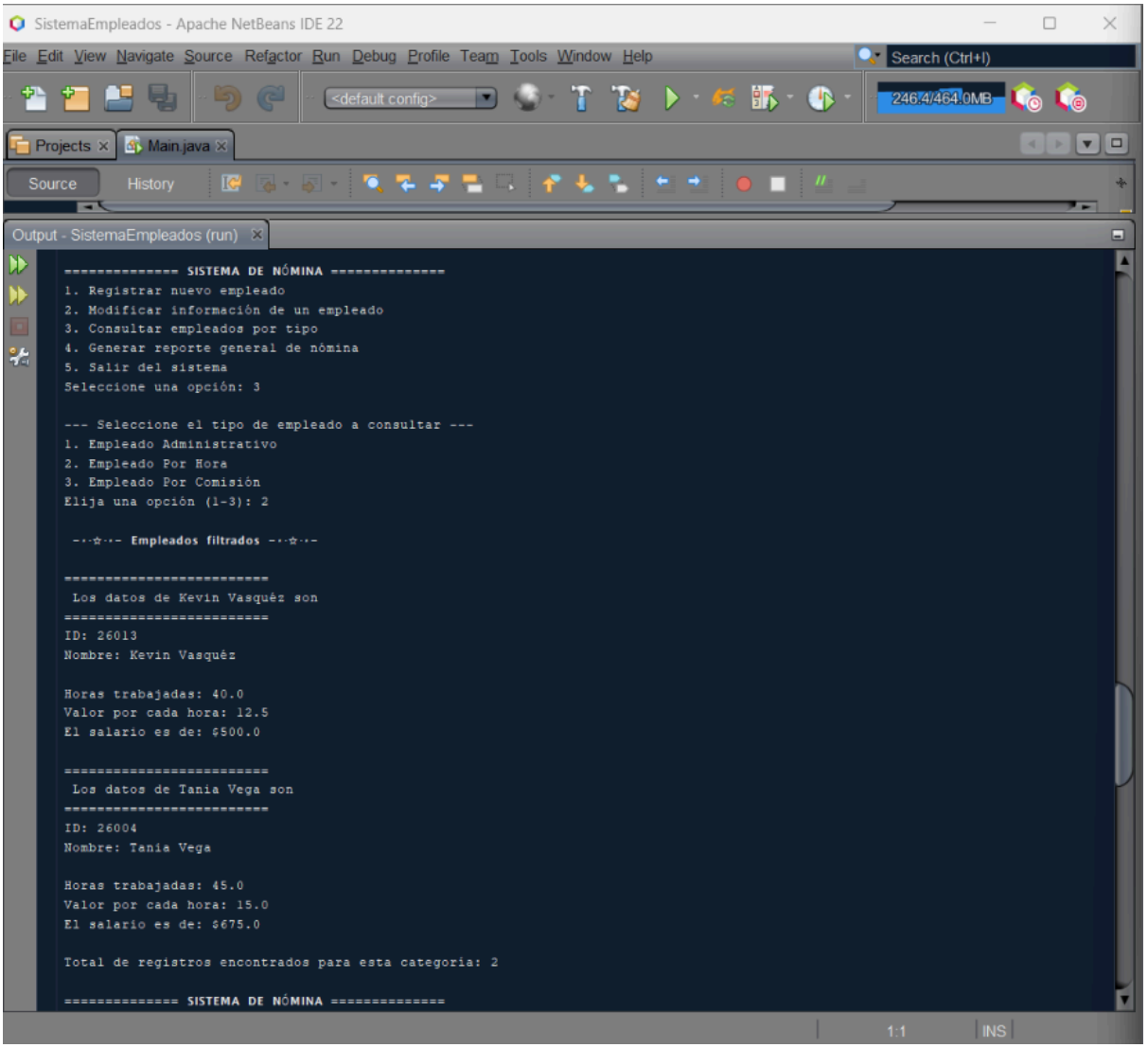
Registro de una cuarta empleada (por hora), y reporte general de nómina actualizado

El reporte lista cuatro empleados: Angel Hércules (ID 26008, salario 1500.00), Kevin Vasquèz (ID 26013, salario 500.00), Roberto Fernández (ID 26003, salario 700.00) y Tania

Vega (ID 26004, salario 675.00). El resumen indica 4 empleados en total (1 administrativo, 2 por hora y 1 por comisión) y un pago total de $3375.00.

Figura 5.

Registro de una empleada más como prueba.



Nota. Captura de pantalla de consola de nuestro código.

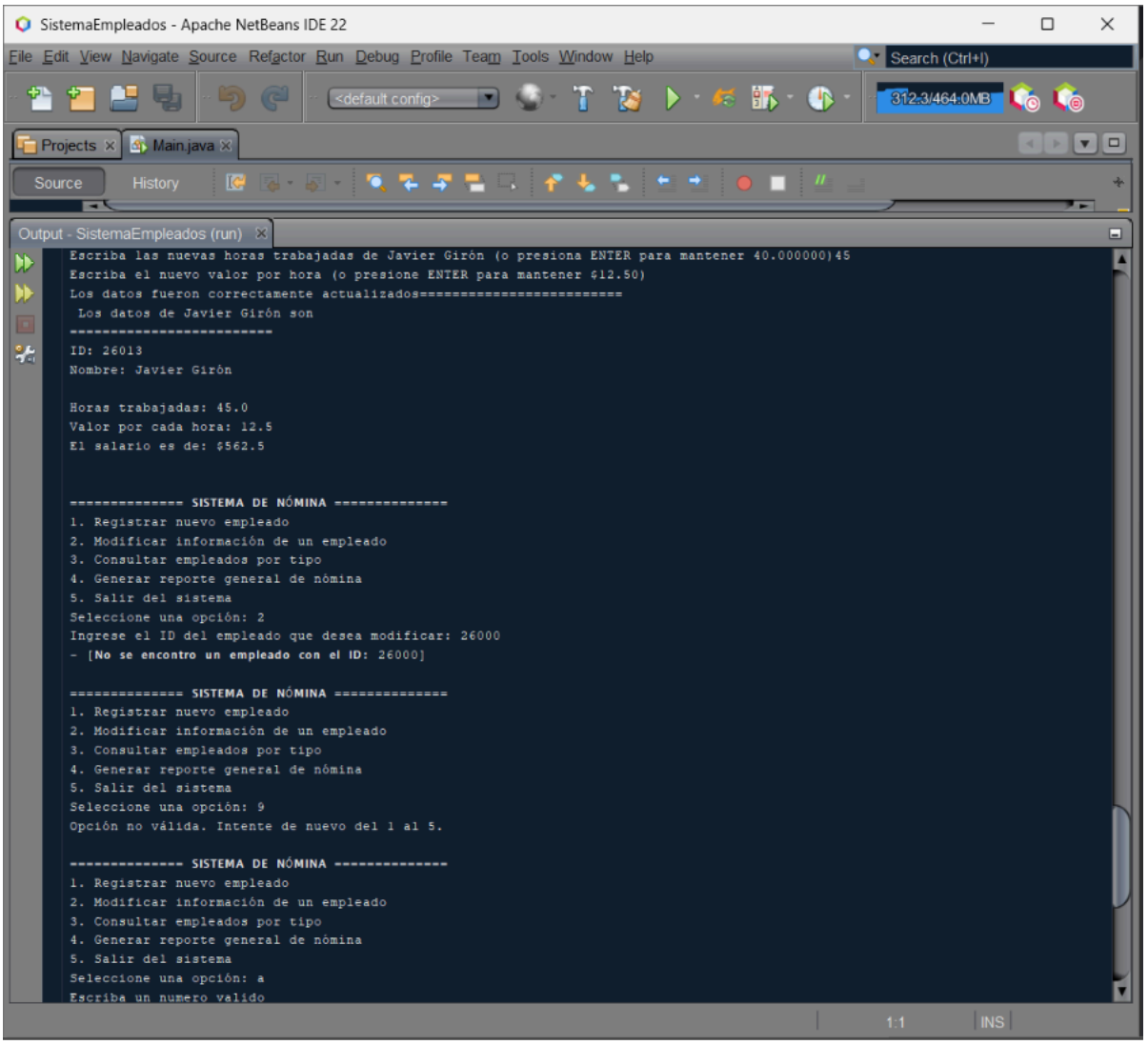
Consulta de empleados por tipo «Por Hora»(Prueba #5)

Aparecen 2 empleados: Kevin Vasquèz (ID 26013, 40.0 horas a $12.5 por hora, salario $500.0) y Tania Vega (ID 26004, 45.0 horas a $15.0 por hora, salario $675.0). La

consulta termina con «Total de registros encontrados para esta categoría: 2» y el sistema vuelve al menú principal

Figura 6.

Consulta de empleados por tipo (por hora) como pueba.



Nota. Captura de pantalla de consola obtenida de nuestro código.

Modificación de un empleado por ID y validación de entradas del menú

Modificación por ID(prueba #6): En la parte superior se ve el final de la opción 2 (Modificar información de un empleado) aplicada al empleado con ID 26013. El sistema pide las nuevas horas trabajadas («o presiona ENTER para mantener 40.000000») y se ingresan 45. Para el valor por hora se presiona ENTER sin escribir nada, y el sistema conserva

$12.50. Luego muestra «Los datos fueron correctamente actualizados» y los datos del empleado: ahora aparece como Javier Girón (antes Kevin Vasquèz), con 45.0 horas, valor por hora de 12.5 y salario de $562.5.

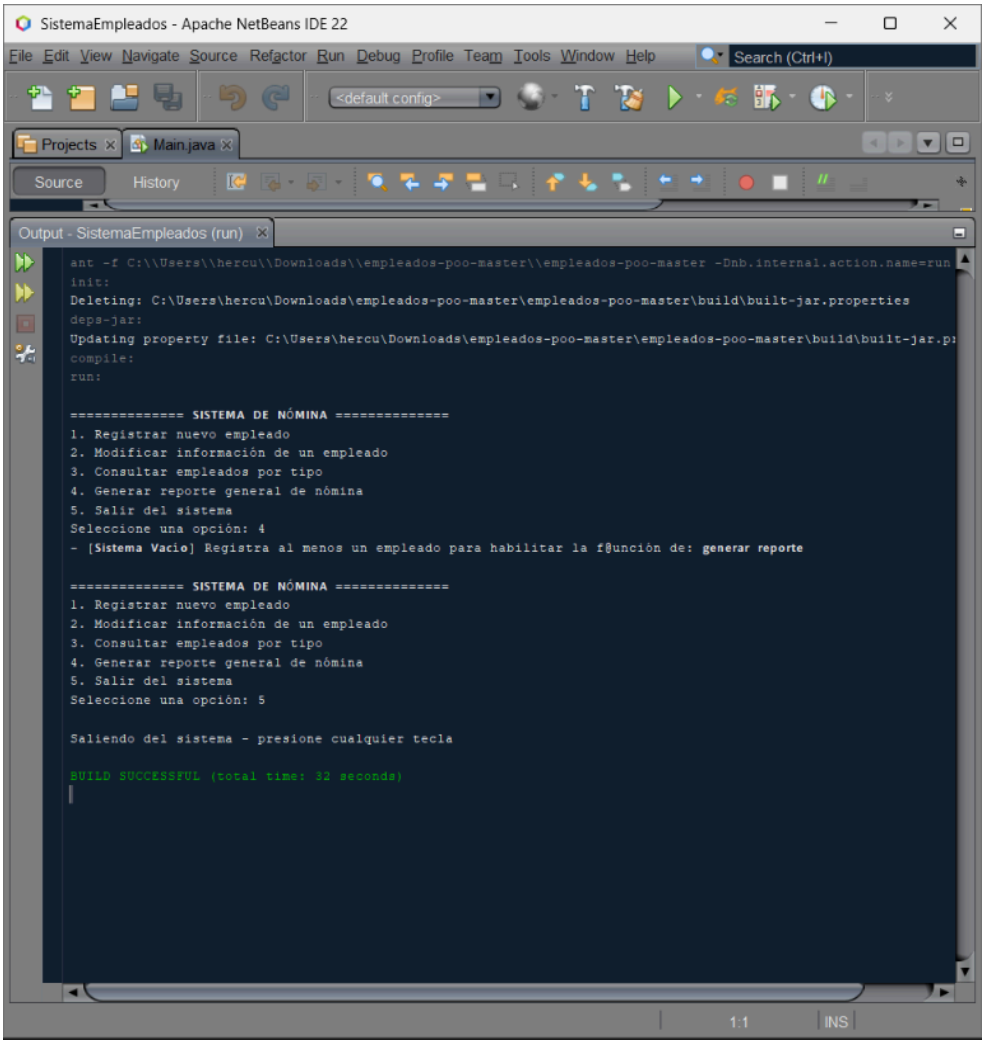
ID inexistente(Prueba #7): Al elegir de nuevo la opción 2 e ingresar el ID 26000, que no está registrado, el sistema responde «No se encontro un empleado con el ID: 26000» y regresa al menú sin errores.

Opción fuera de rango(Prueba #8): Al escribir 9 en el menú principal, el sistema muestra «Opción no válida. Intente de nuevo del 1 al 5.» y repite el menú.

Entrada no numérica(Prueba #9): Al escribir la letra «a», el sistema muestra «Escriba un número valido» y repite el menú, sin cerrarse.

Figura 7.

Modificación de un empleado por ID y validación de entradas del menú como prueba.



Nota. captura de pantalla de consola obtenida de nuestro código.

Reporte con el sistema vacío y salida

El sistema no muestra un reporte vacío ni falla(Prueba #10): responde con el aviso «[Sistema Vacío] Registra al menos un empleado para habilitar la función de: generar reporte» y vuelve al menú. Después se elige la opción 5 y el programa termina con «Saliendo del sistema - presione cualquier tecla» y «BUILD SUCCESSFUL»(Prueba #12).

Conclusiones

Principales conocimientos adquiridos

Mediante el desarrollo de este proyecto de investigación fuimos capaces de entender y aplicar satisfactoriamente el concepto de herencia, y que esta misma tiene como objetivo el reutilizar código y especializarse en una clase que derive de otra, siempre que entre la subclase y la superclase exista una relación conceptual de «es-un» (Meyer, 1996). Se aprendió a modelar una clase abstracta: Empleado, define los datos comunes que todos los empleados deben tener entre sus atributos (idEmpleado y nombre), y además calcularSalario() y actualizarDatos() son métodos que las clases hijas o subclases deben implementar, aunque la clase principal no dicta cómo se debe implementar, las clases de EmpleadoAdministrativo, EmpleadoPorHora y EmpleadoPorComision tienen su propio proceso, pero cada una debe implementar los métodos que indica la clase padre. Además de aplicar la sobrescritura mediante @Override, el uso de super en los constructores de cada clase y el encapsulamiento mediante atributos privados, cuyo uso está restringido y controlado por métodos públicos. Además, se entendió cómo el polimorfismo y el enlace dinámico permiten que una misma operación se ejecute de forma distinta según el tipo real del objeto (Saini, 2009). Durante el desarrollo, el equipo también trabajó de forma colaborativa con Git y GitHub.

Cumplimiento de los objetivos planteados.

Consideramos que los objetivos principales planteados en la guía de desarrollo fueron cumplidos de manera satisfactoria, llámese el diseño e implementación de un sistema de administración de empleados por jerarquía de clases de comportamiento polimórfico, en cuanto a la investigación sobre los fundamentos teóricos de herencia y polimorfismo concordamos que fue desarrollada y completada acorde a la guía. Y también se elaboraron los diagramas UML de casos de uso y de clases, coherentes con los requerimientos del caso de estudio, y se implementó la solución en Java (JDK 21, NetBeans 22) y se definieron casos de prueba representativos para verificar su funcionamiento, cuyos resultados se presentan en la sección de pruebas.

Resultados obtenidos mediante la aplicación de los conceptos investigados.

La aplicación final permite al usuario registrar tres tipos de empleados (de tipo Administrativo, empleado por Hora y empleado por comisión) modificar su información, consultarlos por tipo y generar un reporte de nómina con el total a pagar, rechazando IDs duplicados y validando las opciones del menú. La totalidad de los empleados registrados se almacena en una lista de tipo ArrayList<Empleado>, y el gestor calcula los salarios sin distinguir el tipo de cada objeto, porque el polimorfismo elige la implementación correcta.

Reflexión sobre la importancia de los conceptos estudiados en el desarrollo de soluciones de software.

Tanto la herencia como el polimorfismo son fundamentales: la herencia permite reutilizar y especializar código, mientras que el polimorfismo permite que un mismo concepto/función/comportamiento genere una respuesta distinta entre varios contextos, permitiendo así que un mismo método pueda devolver resultados distintos dependiendo de la clase a la que se le aplique dicho método, a nivel de código, una superclase no especifica cómo se debe comportar el método, pero sí obliga a las demás subclases a implementar

dicho método. Pero el concepto de polimorfismo no es exclusivo del código ya que a nivel biológico permite a las especies adaptarse a entornos cambiantes (Instituto Nacional de Investigación del Genoma Humano (NHGRI)., n.d.). También a nivel de farmacología, ayuda a estudiar y a entender por qué ciertos grupos de pacientes responden de manera diferente a algunos medicamentos según su perfil genético, por ende, el término polimorfismo también aparece en otras áreas, aunque con un significado distinto al que tiene en programación.

Referencias Bibliográficas

Danforth S, Tomlinson C. (1988) Type theories and object-oriented programming. ACM Computing Surveys.

https://doi.org/10.1145/62058.62060

Gotterbarn D, Henry J. (1996). Coupling and cohesion in object-oriented design and coding. Proceedings of the 1996 ACM 24th Annual Conference on Computer Science - CSC '96. ACM Press; 1996:149.

https://doi.org/10.1145/228329.304561

Kilian MF. (1991) A note on type composition and reusability. ACM SIGPLAN OOPS Messenger.

https://doi.org/10.1145/122242.122245

Kobylinski K, McLeod RD, Shome T. (1997). Design reuse in object oriented software architecture. In: 1997 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing, PACRIM. Vol 2. IEEE; 1997:993-997.

https://doi.org/10.1109/pacrim.1997.620427

Meyer B. (1996). The many faces of inheritance: a taxonomy of taxonomy. Computer. 1996;29(5):105-108.

https://doi.org/10.1109/2.494093

Riehle D. (1997). Composite design patterns. In: Proceedings of the 12th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications..

https://doi.org/10.1145/263698.263739

Robillard MP. (2022). Introduction to Software Design with Java. Springer International Publishing.

https://doi.org/10.1007/978-3-030-97899-0

Saini DK. (2009). Testing polymorphism in object oriented systems for improving software quality. ACM SIGSOFT Software Engineering Notes.
https://doi.org/10.1145/1507195.1507212

Snyder A. (1986). Encapsulation and inheritance in object-oriented programming languages. ACM SIGPLAN Notices.

https://doi.org/10.1145/960112.28702

Instituto Nacional de Investigación del Genoma Humano. (2026, 9 de octubre). *Polimorfismo*. Genome.gov.

https://www.genome.gov/es/genetics-glossary/Polimorfismo

.

Anexos

Enlace al video de exposición y demostración del caso de estudio.

Herencia y Polimorfismo en el diseño de Sistemas Equipo 6